Intune Android Management API (AMAPI) Migration — Legacy Play EMM API → AMAPI

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary 4

Solution Overview 4

What this solution does . . . 4

What AMAPI is, and how it differs from the legacy Play EMM API . . . 5

The enrollment methods affected . . . 5

The web / account-driven enrollment path . . . 6

The managed Google Play binding still underpins everything . . . 6

Benefits . . . 6

Limitations and boundaries . . . 6

Deployment model . . . 7

Requirements 7

R1. A healthy managed Google Play binding (the connection you are migrating) . . . 8

R2. A current inventory of the Android estate and enrollment models . . . 8

R3. Confirmed current AMAPI migration guidance and status for the tenant . . . 8

R4. A verified end-user / re-enrollment impact statement (per enrollment model) . . . 9

R5. Web / account-driven enrollment readiness decision (where applicable) . . . 9

R6. Enrollment profiles, app assignments, and Conditional Access reviewed . . . 9

R7. A pilot cohort per enrollment model and change control . . . 9

R8. Microsoft Graph PowerShell for automation . . . 9

Global Requirements — Microsoft Intune Android (Android Enterprise) Solutions 10

1. Microsoft 365 tenant with Intune and Android Enterprise . . . 10

2. The managed Google Play binding (account enterprise OR Google Workspace) . . . 10

3. Administrative access (least-privilege Intune roles) . . . 11

4. Enrollment method decision and Graph/PowerShell toolchain . . . 11

5. Network reachability to Google & Microsoft endpoints . . . 11

6. A pilot scope and change control . . . 12

Implementation 12

Phase 0 — Assess (read-only; change nothing) . . . 12

Phase 1 — Design (plan the wave; decide the enrollment path) . . . 13

Phase 2 — Configure (prepare, don't migrate yet) . . . 13

Phase 3 — Pilot (migrate a small cohort; OBSERVE the impact) . . . 14

Phase 4 — Validate (prove the three outcomes) . . . 14

Phase 5 — Expand (wave by wave) . . . 15

Order of operations (must not be reordered) . . . 15

Testing & Validation 15

V1. Binding is healthy (R1) . . . 16
V2. Devices manage under AMAPI (the core outcome) . . . 16
V3. End-user / re-enrollment impact matches the documented statement (R4) . . . 16
V4. Managed Google Play apps deliver . . . 16
V5. Managed Google account adds / is present . . . 16
V6. Devices remain compliant and Conditional Access passes . . . 16
V7. Wave repeatability . . . 17
Troubleshooting 17
Decision tree (start at the top) . . . 17
§1 — Device missing / not checking in after migration . . . 17
§2 — "Can't add a Google account" during (re-)enrollment . . . 18
§3 — Device present but non-compliant / Conditional Access blocks the user . . . 18
§4 — Managed Google Play apps did not (re)install . . . 18
§5 — Device re-enrolled or work profile reset **unexpectedly** . . . 18
§6 — Whole cohort / app-sync looks wrong (managed Google Play binding) . . . 19
§7 — Admin can't see/select the wave group or devices (custom Intune RBAC) . . . 19
§8 — Everything checks and it still fails (escalation) . . . 19
Best Practices 19
Verify, never assume (the cardinal rule) . . . 19
Estate and wave design . . . 20
Protect the managed Google Play binding . . . 20
End-user communication . . . 20
Change control, validation, and rollback . . . 20
Tooling and least privilege . . . 21
Documentation and evidence . . . 21
Deliverables Summary 21
Documentation . . . 21
PowerShell (Microsoft Graph) tools . . . 22
Appendix A — PowerShell Scripts 24

Executive Summary

Outcome. Intune's Android Enterprise management moved from the **legacy Google Play EMM API** to the **Android Management API (AMAPI)** deliberately and with low disruption — the estate assessed, the migration waved, the end-user and **re-enrollment** impact **verified rather than assumed**, and each cohort validated to confirm devices manage under AMAPI, Google accounts add, and managed Google Play apps still deliver.

The problem it solves. Microsoft is transitioning Intune's Android Enterprise capabilities onto AMAPI. Teams need three things they usually lack: a clear **migration approach**, an honest read on the **end-user impact**, and certainty about **whether devices re-enroll**. The rollout is staged and **actively evolving**, so the biggest risk is acting on stale assumptions — communicating the wrong outcome, resetting a kiosk unexpectedly, or breaking app delivery — because someone asserted a behavior from memory instead of verifying it.

Approach. A pilot-first playbook built around a single discipline: **verify every version-specific behavior against current Microsoft guidance and your own tenant.** It assesses the Android estate and enrollment models, plans a migration wave, decides the enrollment path per model (including a readiness check for the newer **web / account-driven enrollment**), then **pilots each enrollment model to observe the real re-enrollment / user impact before expanding** — turning the impact statement from a guess into documented fact. Five Microsoft Graph PowerShell tools inventory the estate, plan waves, check web-enrollment readiness, and produce a post-migration validation report — all read-only or `-WhatIf` by default, least-privilege, no stored secrets.

Why it is low-risk. Everything is **pilot-first** with explicit pass/fail gates — *manage under AMAPI → accounts add → apps deliver → compliant* — and a **halt-and-hold** rollback that keeps un-migrated devices on the current path and never disturbs already-validated ones. Critically, the solution **refuses to publish a timeline or re-enrollment outcome it cannot verify**: every such behavior is flagged for confirmation against Microsoft's live guidance and proven in the pilot. The result is a repeatable, evidence-backed migration that stays current as AMAPI evolves — the estate migrated with eyes open, not on assumptions.

Solution Overview

What this solution does

Microsoft is moving Intune's Android Enterprise management from the **legacy Google Play EMM API** to the **Android Management API (AMAPI)**. This solution is the migration playbook for that transition: you **assess** the organization's current Android estate and enrollment models, **plan** a migration wave, **execute** the move (including the newer **web / account-driven enrollment** path where it applies), **validate** that devices manage under AMAPI, that Google accounts add, and that apps still deliver — and you handle the common post-migration failures.

The single question every team asks — *"do our devices re-enroll, and what does the end user see?"* — is treated as a **first-class risk**, not a footnote. Because the AMAPI transition, web/account-driven enrollment, and re-enrollment behavior are **actively evolving**, this playbook **flags every version-specific behavior for verification against current Microsoft guidance and your own tenant** rather than asserting a fixed timeline or outcome from memory.

> ***Verify-first discipline (state this to stakeholders).*** *The AMAPI rollout is staged and changing. Timelines, which enrollment methods move when, the exact portal labels, and — most importantly — **whether any given device re-enrolls** are things you must confirm against Microsoft's current Android Management API / "What's new" documentation and by testing in your pilot. Where this document says **"verify against current Microsoft guidance,"** treat it as a hard gate, not advice.*

What AMAPI is, and how it differs from the legacy Play EMM API

Android Enterprise is Google's framework for managing Android in the enterprise (managed Google Play, work profiles, fully managed and dedicated devices). An EMM such as Intune talks to Google through an API to provision the managed Google Play enterprise, push apps, and drive enrollment and policy.

• **Legacy Play EMM API** — the original Google Play EMM API that Intune has historically used to broker Android Enterprise management.

• **Android Management API (AMAPI)** — Google's **newer, consolidated** management API. Microsoft is transitioning Intune's Android Enterprise capabilities onto AMAPI. Much of the change is a **backend / service-side** move: the management *plane* changes, while the day-to-day admin concepts (work profile, fully managed, dedicated/COSU, managed Google Play apps) remain.

What this means in practice, at the level you can rely on:

• The **enrollment models are conceptually the same** — personally-owned **work profile (BYOD)**, **fully managed**, **corporate-owned work profile**, and **dedicated / COSU (kiosk)** — but the **enrollment mechanics and available options can shift** as methods move to AMAPI. **Verify the current enrollment options for each model in your tenant.**

• A newer **web-based / account-driven enrollment** experience is part of the modern Android enrollment direction. **Its availability, the exact user flow, and which device types it applies to are evolving — verify against current Microsoft guidance before you design around it.**

• **Whether a device re-enrolls is the key unknown and is version-dependent.** Do **not** promise end users "no re-enrollment" (or "you must re-enroll") until you have confirmed the behavior for your specific enrollment model, device type, and the current state of the AMAPI rollout. **Test it in the pilot and read it off Microsoft's current guidance.**

The enrollment methods affected

Enrollment model	What it is	Migration sensitivity
Work profile (BYOD)	Personally-owned device with a managed work profile	High-visibility to end users; re-enrollment / re-consent behavior **must be verified**
Fully managed	Corporate-owned, fully managed device	Corporate-owned; coordinate any re-enrollment as a change
Corporate-owned work profile	Corporate device with a separate work profile	As above; verify user-facing impact

Enrollment model	What it is	Migration sensitivity
Dedicated / COSU (kiosk)	Single-purpose / shared kiosk devices, no user affinity	Often no interactive user, but a re-provision can wipe/reset a kiosk — **verify and stage carefully**

For each model, the **user-facing impact** (re-enrollment, re-consent, a wipe of the work profile, or nothing at all) is **exactly what you validate in the pilot before you expand** — and exactly what you communicate to users **only after** you have confirmed it.

The web / account-driven enrollment path

Modern Android enrollment is trending toward a **web-based / account-driven** experience (the user signs in and the enrollment is driven from an account/web flow rather than a QR/token-first setup). This solution includes a **readiness check** for that path — managed Google Play binding present, enrollment profiles configured, network endpoints reachable — but treats the **exact availability and flow as evolving**. **Confirm whether web/account-driven enrollment is available for your enrollment model and tenant against current Microsoft guidance before committing to it in the wave plan.**

The managed Google Play binding still underpins everything

AMAPI does not remove the **managed Google Play** connection — Android Enterprise still depends on it. Per the category **Global Requirements**, exactly one binding model applies to the tenant:

- **Managed Google Play account enterprise** (tenants **without** Google Workspace), or
- **Google Workspace / Cloud Identity ↔ Intune third-party EMM** (tenants that **use** Google Workspace).

The migration assumes this binding is healthy. If it is not, fix the binding first (see the category Global Requirements and the connection solution) before migrating.

Benefits

- **Migrate with eyes open** — the estate is assessed, the wave is planned, and the end-user / re-enrollment impact is **verified**, not assumed.
- **Low disruption** — pilot-first, with explicit pass/fail gates and a documented rollback before any production wave.
- **Evidence-backed** — five Microsoft Graph PowerShell tools inventory the estate, plan waves, check web-enrollment readiness, and produce a post-migration validation report.
- **Current, not stale** — every version-specific behavior is flagged for verification against Microsoft's live guidance, so the plan does not rot as AMAPI evolves.

Limitations and boundaries

- **Not a source of AMAPI timelines.** This solution does **not** publish a Microsoft rollout

timeline or assert re-enrollment behavior — those are read from Microsoft's current guidance and proven in your pilot.

- **No invented cmdlets or portal paths.** Where Microsoft's exact blade label or a Graph property is version-dependent, it is flagged **"verify in your tenant."** The scripts use only documented, read-only Graph reads (plus optional `-WhatIf`-gated group creation for wave planning).
- **OEM and third-party apps at the pattern level.** Samsung Knox, Zebra, and other rugged/kiosk specifics defer to the vendor's documentation; managed Google Play and Graph are authored exactly.
- **Assumes the category prerequisites are met** — a healthy managed Google Play binding, Intune as MDM authority, admin roles, and network reachability (see the Global Requirements).
- **Post-migration deep troubleshooting** cross-references the dedicated **Android enrollment troubleshooting (Google account / AMAPI)** solution for symptom-level "can't add a Google account" / enrollment-failure diagnosis.

Deployment model

This solution follows the standard house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — with explicit pass/fail gates and a documented rollback at each production step:

1. **Assess** — inventory the Android estate and current enrollment models (`AMAPIMIG-Assess-AndroidEstate.ps1`) and confirm the managed Google Play binding health (`AMAPIMIG-Config.ps1`).
1. **Design** — plan the migration wave and define pilot/production cohorts (`AMAPIMIG-New-MigrationWavePlan.ps1`); decide the enrollment path per model, **after** verifying web/account-driven availability (`AMAPIMIG-Test-WebEnrollmentReadiness.ps1`).
1. **Configure** — prepare enrollment profiles, app assignments, and wave groups (safe/`-WhatIf` helpers).
1. **Pilot** — migrate a small cohort; **observe the actual end-user / re-enrollment behavior**.
2. **Validate** — devices manage under AMAPI, accounts add, apps deliver (`AMAPIMIG-Get-PostMigrationValidation.ps1`).
1. **Expand** — promote wave by wave; keep the validation and diagnostic tools on hand.

Requirements

These are the **solution-specific** prerequisites for migrating Intune's Android Enterprise management to the **Android Management API (AMAPI)**. They build **on top of** the category-level **Global Requirements — Microsoft Intune Android** (M365 tenant with Intune + Android Enterprise, the managed Google Play binding, least-privilege admin roles, the enrollment-method decision + Graph toolchain, network reachability, and a pilot scope with change control) — confirm those first; this document does **not** repeat them.

Each requirement is stated as **Description · Why · Verify · Implement-if-missing.**

> ***Version boundary (applies to R3, R4, R5).*** *The AMAPI transition, web/account-driven enrollment, and re-enrollment behavior are* ***actively evolving****. Where a requirement depends on current Microsoft behavior, it says* ***"verify against current Microsoft guidance / your tenant"*** *— that verification is part of satisfying the requirement, not optional.*

R1. A healthy managed Google Play binding (the connection you are migrating)

Description: The tenant's **managed Google Play** connection — either the **account enterprise** model (no Google Workspace) or the **Google Workspace / Cloud Identity ↔ Intune third-party EMM** integration — must be **present and healthy** before migrating. **Why:** AMAPI does not remove managed Google Play; the migration moves the *management plane* while the binding continues to underpin enrollment and app delivery. A broken/expired binding will surface as migration and app-sync failures. **Verify:** Intune admin center → **Devices > Android > Android enterprise** (or **Enrollment**) — confirm the connection status is **connected** and app sync is current. Via Graph (beta): `GET /deviceManagement/androidManagedStoreAccountEnterpriseSettings` — check `bindStatus` and `lastAppSyncDateTime`. `AMAPIMIG-Config.ps1 -HealthSummary` reports it. **Implement:** Repair or re-establish the binding first (see the category Global Requirements and the Android Enterprise connection solution). Do **not** migrate on top of a degraded binding.

R2. A current inventory of the Android estate and enrollment models

Description: A known inventory of enrolled Android devices grouped by **enrollment model** (work profile / fully managed / corporate-owned work profile / dedicated-COSU), ownership, OS/OEM, and last check-in. **Why:** Migration risk and end-user impact differ per enrollment model; you cannot wave-plan or scope communications without knowing the population. **Verify:** Intune → **Devices > All devices** (filter OS = Android), or run `AMAPIMIG-Assess-AndroidEstate.ps1` (read-only) for a breakdown by `deviceEnrollmentType` / `managementAgent` / `ownerType` / `osVersion` / `manufacturer`. **Implement:** Run the assessment and reconcile against your CMDB before designing waves.

R3. Confirmed current AMAPI migration guidance and status for the tenant

Description: The **current** Microsoft guidance on the AMAPI transition — what has moved, what is pending, any admin action Microsoft requires, and any tenant-level migration status Microsoft surfaces — **read at the time you plan the migration**. **Why:** The rollout is staged and changing; the correct sequence and whether admin action is needed are version-dependent and **must not** be assumed from memory. **Verify:** Review Microsoft's current **Android Management API / Intune "What's new" and Android Enterprise** documentation and any **Message center** posts for the tenant. Confirm whether Intune presents an AMAPI migration status or required action in the admin center. **Implement (if unknown):** Do not proceed on assumptions — establish the current guidance first. This requirement is satisfied only when the team can state, in writing, what the current rollout requires. **Verify against current Microsoft guidance / your tenant.**

R4. A verified end-user / re-enrollment impact statement (per enrollment model)

Description: A written, **tenant-verified** statement of what each enrollment model's users will experience during migration — **no change**, **re-consent**, **re-enrollment**, or a **work- profile reset/wipe** — confirmed by pilot observation and current Microsoft guidance. **Why:** This is the highest-impact unknown. Communicating the wrong outcome (or none) erodes trust and can cause data loss on dedicated devices. It **cannot** be asserted from memory because it is version- and model-dependent. **Verify:** Confirm the behavior in the **pilot** (Phase Pilot in Implementation) and against current Microsoft guidance for each model you operate. Capture screenshots of the actual user flow. **Implement (if unknown):** Treat "unknown" as a blocker for that model's production wave. Pilot it, verify it, write it down, **then** communicate. **Verify against current Microsoft guidance / your tenant.**

R5. Web / account-driven enrollment readiness decision (where applicable)

Description: A decision — informed by a readiness check — on whether the migration will adopt the newer **web / account-driven enrollment** path for any model, based on its **current availability** for your tenant and enrollment model. **Why:** Web/account-driven enrollment is part of the modern direction but its availability and flow are evolving; designing a wave around it before confirming availability creates rework. **Verify:** `AMAPIMIG-Test-WebEnrollmentReadiness.ps1` (read-only) checks the binding, enrollment profiles, and prerequisite signals; then **confirm the method's availability for your enrollment model against current Microsoft guidance. Implement:** If available and desired, include it in the wave design; if not (or unverified), plan the migration on the currently-supported enrollment method and revisit later. **Verify against current Microsoft guidance / your tenant.**

R6. Enrollment profiles, app assignments, and Conditional Access reviewed

Description: The current **Android enrollment profiles** (e.g. device-owner enrollment profiles / tokens for dedicated & fully managed), **managed Google Play app assignments**, and any **Conditional Access / compliance policies** targeting Android are reviewed and understood before migration. **Why:** A migration that changes enrollment must preserve app delivery and not inadvertently lock users out via a compliance/CA policy that re-evaluates on re-enrollment. **Verify:** Intune → **Devices > Android > Enrollment** (profiles/tokens), **Apps > Android**, and **Endpoint security / CA**. Via Graph (beta): `androidDeviceOwnerEnrollmentProfiles`, `deviceAppManagement/mobileApps` (androidManagedStoreApp), and compliance/CA policies. **Implement:** Document the current assignments so post-migration validation (apps deliver, compliant, CA passes) has a known baseline.

R7. A pilot cohort per enrollment model and change control

Description: A **named pilot cohort for each enrollment model** you operate (a few work-profile users, a fully-managed device, a spare dedicated/COSU device), plus a **change window** — because AMAPI-migration changes are **high-impact and may affect existing devices**. **Why:** The house model is pilot-first; re-enrollment/impact behavior must be proven per model on throwaway/representative devices before any production wave. **Verify:** Confirm the pilot devices/users exist per model and a change window is approved. **Implement:** Recruit pilot users, set aside spare corporate/dedicated devices, and agree change control before touching production.

R8. Microsoft Graph PowerShell for automation

Description: The **Microsoft.Graph** PowerShell SDK (v2+), modern-auth, for the five tools in this solution. **Why:** The scripts inventory the estate, plan waves, check readiness, and validate — all via Microsoft

Graph, read-only or `-WhatIf` by default. **Verify:** `Get-Module -ListAvailable Microsoft.Graph.Authentication` returns a module; `Connect-MgGraph` succeeds interactively. **Implement:** `Install-Module Microsoft.Graph -Scope CurrentUser`. Consent the **least** scopes the scripts request — read tools use `DeviceManagementManagedDevices.Read.All`, `DeviceManagementServiceConfig.Read.All`, `DeviceManagementConfiguration.Read.All`, `DeviceManagementApps.Read.All`; the wave-planner requests `Group.ReadWrite.All` **only** when you opt into creating wave groups (and only with `-Commit`). No app secrets are stored — all scripts use interactive/delegated `Connect-MgGraph`.

Prerequisite gate: Do not begin a production wave until **R1–R4, R6, R7** are satisfied (and **R5** where web/account-driven enrollment is in scope). R3 and R4 in particular are only satisfied once the current Microsoft guidance and the pilot-observed impact are **written down** — migrating on assumptions about re-enrollment is the failure this solution exists to prevent.

Global Requirements — Microsoft Intune Android (Android Enterprise) Solutions

These requirements apply to **every** solution in the **Android** category (Android Enterprise connection & managed Google Play, Google Workspace ↔ Intune EMM integration, the Android Management API (AMAPI) migration, dedicated/COSU & work-profile enrollment, app management, and enrollment troubleshooting). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Android solution).*** *The **Microsoft Intune admin center** (intune.microsoft.com), **Microsoft Graph**, the **managed Google Play** enrollment surface, and the **Google Admin console** (for Google Workspace / Cloud Identity integration) are authored at exact, current click-by-click / cmdlet detail from Microsoft's and Google's public documentation. The **Android Management API (AMAPI)** transition (Intune's move off the legacy Play EMM API) and **web-based / account-driven enrollment** are **actively evolving** — steps, defaults, and whether a change triggers **re-enrollment** are **cited or flagged "verify against current Microsoft guidance / your tenant"**, never asserted from memory or invented. **OEM specifics** (Samsung Knox, Zebra, and other rugged/kiosk vendors) and **third-party apps** are integrated at the pattern level; their consoles are deferred to the vendor's docs. Automation is **PowerShell** (Microsoft Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with Intune and Android Enterprise

- **Description:** A Microsoft 365 tenant with **Microsoft Intune** and Android Enterprise available,

and the licensing the capability requires.

- **Why required:** All Android policy, enrollment, and app configuration is performed in Intune.
- **How to verify:** Sign in to the Intune admin center; confirm Android platform availability.
- **How to implement if missing:** Provision/assign Intune licensing before starting.

2. The managed Google Play binding (account enterprise OR Google Workspace)

• **Description:** Intune connected to **managed Google Play** — either the **managed Google Play account enterprise **(for tenants WITHOUT Google Workspace) or the** Google Workspace / Cloud Identity ↔ Intune third-party EMM** integration (for tenants that use Google Workspace). One and only one model applies per tenant.

• **Why required:** Android Enterprise enrollment and app delivery depend on the managed Google Play connection; the two binding models are mutually exclusive and must be chosen deliberately.

• **How to verify:** Intune admin center → **Devices > Android > Android enterprise (or Enrollment)** — confirm the connection status and which model is in place.

• **How to implement if missing:** Establish the correct binding (this is often the first solution in the line) before enrolling devices.

3. Administrative access (least-privilege Intune roles)

• **Description:** Access with the least-privilege role the solution names — typically **Intune Administrator**, with** Global Administrator** only where genuinely required (e.g. the initial managed Google Play / Google Workspace binding).

• **Why required:** Enrollment, connector, and app changes require the corresponding role.

• **How to verify:** Confirm the role in the M365 admin center / Entra roles.

• **How to implement if missing:** Have a Global Admin assign the least-privilege role under change control.

4. Enrollment method decision and Graph/PowerShell toolchain

• **Description:** A decided **enrollment method** — personally-owned **work profile** (BYOD), **fully managed**, corporate-owned work profile**, or** dedicated / COSU (kiosk) **— plus a workstation with** Microsoft Graph PowerShell** for automation.

• **Why required:** Profiles, restrictions, and app assignment differ by enrollment method; the provided scripts run through Graph.

• **How to verify:** Confirm the target enrollment method(s) with the device owners; `Connect-MgGraph` succeeds.

• **How to implement if missing:** Choose the enrollment method and install Graph PowerShell before starting.

5. Network reachability to Google & Microsoft endpoints

• **Description:** Devices and the enrollment path can reach the required **Google (managed Google Play, Android) **and** Microsoft (Intune, Entra)** service endpoints (no proxy/filter blocks the enrollment or account sign-in).

• **Why required:** Enrollment, managed Google Play account provisioning, and adding a Google account all

require reachability; blocked endpoints are a common cause of "can't add a Google account" failures.

• **How to verify:** Confirm the required endpoints are allowed on the device network.

• **How to implement if missing:** Allow the documented Google/Microsoft endpoints before enrolling.

6. A pilot scope and change control

• **Description:** A **pilot group / small device set** to validate on before fleet-wide rollout, and a

change-control process for production changes (enrollment-model, connector, and AMAPI-migration changes are high-impact and may affect existing devices).

• **Why required:** Every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**

model; the pilot proves the design and protects the production fleet.

• **How to verify:** Confirm a named pilot scope and an approval/change window are available.

• **How to implement if missing:** Create a pilot scope and agree change control before production changes.

*The deployment model across every Android solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **PowerShell** (Microsoft Graph) — never with hardcoded credentials or secrets. Where the AMAPI transition or enrollment behavior is version-dependent, verify against current Microsoft guidance rather than assuming.*

Implementation

This is the ordered **AMAPI migration** procedure: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback at each production phase. It is **pilot-first** because AMAPI-migration and enrollment-model changes are high-impact and **may affect existing devices**.

> ***Standing rule for the whole procedure.*** *Every step that depends on the AMAPI rollout — sequence, enrollment-method options, web/account-driven availability, and especially **whether a device re-enrolls** — is a **"verify against current Microsoft guidance / your tenant"** gate. Do not communicate an end-user outcome you have not observed in the pilot and confirmed in Microsoft's current documentation. This procedure names the *artifacts and order*; it does **not** assert Microsoft's timeline or re-enrollment behavior.*
>
> ***Naming convention*** *(`AMAPIMIG – <Model> – <Stage>`), reused in Testing and the scripts: - Wave groups:* `AMAPIMIG – WorkProfile – Pilot` → … – `Wave1` → … – `Prod` *(and per model:* `FullyManaged`, `CorpWorkProfile`, `Dedicated`*). - CSV plan:* `AMAPIMIG-WavePlan.csv`*.*

Phase 0 — Assess (read-only; change nothing)

Step 0.1 — Confirm the binding is healthy (R1). Run `AMAPIMIG-Config.ps1 -HealthSummary`. It reports the managed Google Play binding state (`bindStatus`, `lastAppSyncDateTime`) and connection model. If the binding is degraded, **stop** and repair it first.

Step 0.2 — Inventory the estate (R2). Run `AMAPIMIG-Assess-AndroidEstate.ps1`. It returns Android devices grouped by `deviceEnrollmentType` (work profile / fully managed / corp work profile / dedicated-COSU), `managementAgent`, `ownerType`, `osVersion`, `manufacturer`, and last check-in — a per-model population count and a stale-device list.

> ***On telling "legacy Play EMM" from "AMAPI" per device.*** *There is* ***no single documented Graph property*** *that cleanly labels a device "managed under AMAPI" vs "legacy Play EMM." The assessment reports what Graph exposes and the* ***tenant-level*** *binding/migration signals; the authoritative legacy-vs-AMAPI status is what* ***Microsoft surfaces in the admin center / current guidance****.* ***Verify the migration status there — do not infer it from an invented property.***

Step 0.3 — Read the current AMAPI guidance (R3). Review Microsoft's current Android Management API / "What's new" documentation and the tenant **Message center**. Write down: what has moved, what is pending, and any **required admin action**.

Gate 0 (pass/fail): Binding healthy **and** the team can state, in writing, the current AMAPI status and any required action. Do not design waves until both are true.

Phase 1 — Design (plan the wave; decide the enrollment path)

Step 1.1 — Generate the wave plan. Run `AMAPIMIG-New-MigrationWavePlan.ps1` (read-only by default). It buckets the assessed devices into **Pilot** → **Wave1** → **Wave2** → **Prod** cohorts per enrollment model, applying your criteria (model, ownership, OS floor, staleness), and writes `AMAPIMIG-WavePlan.csv`. Least-risk models/cohorts go first; dedicated/COSU and BYOD work-profile (highest user visibility) are staged deliberately.

Step 1.2 — Check web/account-driven enrollment readiness (R5). Run `AMAPIMIG-Test-WebEnrollmentReadiness.ps1`. It confirms the binding, enrollment profiles, and prerequisite signals, then **prompts you to confirm the method's availability for each enrollment model against current Microsoft guidance**. Decide per model whether the wave adopts web/account-driven enrollment or the currently-supported method.

Step 1.3 — Draft the per-model impact statement (R4). For each model, record the **expected** end-user outcome (no change / re-consent / re-enrollment / work-profile reset) as a **hypothesis to be proven in Pilot** — explicitly marked "unverified" until Phase Pilot confirms it.

Gate 1 (pass/fail): A wave plan exists, the enrollment path per model is decided (and its availability verified), and a draft impact statement per model is written (even if "to be verified").

Rollback 1: Design artifacts only — discard the plan/CSV. No production impact.

Phase 2 — Configure (prepare, don't migrate yet)

Step 2.1 — Prepare wave groups. Create the Assigned security groups from the plan. Either in Entra (**Groups** → **New group** → **Security, Assigned**) or with `AMAPIMIG-New-MigrationWavePlan.ps1 -CreateGroups` (preview) then `-CreateGroups -Commit` to create them. Start each group **scoped to**

its Pilot cohort only.

Step 2.2 — Confirm enrollment profiles & app assignments (R6). Verify the enrollment profiles (device-owner profiles/tokens for dedicated & fully managed) and **managed Google Play app assignments** that the migrated cohort will rely on are in place and target the wave groups (or their users). Record the baseline so validation can prove "apps still deliver."

Step 2.3 — Confirm Conditional Access / compliance won't lock users out. Review Android CA / compliance so a re-enrollment (if it occurs) does not strand users behind a grace-less policy during cutover.

Gate 2 (pass/fail): Pilot wave groups exist and are correctly scoped; enrollment profiles and app assignments are confirmed against the wave groups; CA/compliance reviewed.

Rollback 2: Delete the newly created wave groups (nothing is migrated yet), revert any assignment changes to the recorded baseline.

Phase 3 — Pilot (migrate a small cohort; OBSERVE the impact)

Do this for **one model at a time**, lowest-visibility first.

Step 3.1 — Execute the pilot migration per current guidance. Perform the migration action Microsoft's **current** guidance specifies for that enrollment model (an admin action in the admin center, an enrollment-profile change, and/or device-side re-enrollment as directed). **Follow the current documented steps — this procedure does not substitute for them.**

Step 3.2 — Observe the real end-user experience (satisfies R4). On the pilot devices, **watch what actually happens**: does the device stay enrolled, re-consent, **re-enroll**, or reset the work profile? Capture screenshots and timing. **Update the impact statement from "unverified" to the observed, verified outcome for that model.**

Step 3.3 — Confirm the device manages under AMAPI. Confirm in the admin center that the pilot device reflects the post-migration (AMAPI) management state Microsoft's current guidance describes.

Gate 3 (pass/fail):

- The pilot device **manages under AMAPI** (per Microsoft's current status surface), **and**
- the **end-user impact is now observed and documented** for that model, **and**
- the device is **compliant** and reachable.

Only a full pass promotes the model to production waves.

Rollback 3 (before wide impact): If the pilot fails or the user impact is unacceptable, **halt that model's migration**, keep the remaining devices on the current (pre-migration) path, and raise the finding. Because you piloted a small cohort, exposure is limited. For device-level recovery of a failed pilot device, re-enroll it clean per current guidance and cross-reference the **Android enrollment troubleshooting (Google account / AMAPI)** solution.

Phase 4 — Validate (prove the three outcomes)

Step 4.1 — Run the post-migration validation report. Run `AMAPIMIG-Get-PostMigrationValidation.ps1 -GroupName "AMAPIMIG – WorkProfile – Pilot"`. It reports, for the cohort: device present & checking in, **compliant**, **managed-app install status** (apps deliver), and account/ownership signals — against the Phase 2 baseline.

Step 4.2 — Confirm accounts add and apps deliver. Specifically confirm a managed Google Play app **installs** on a migrated device and (where relevant) the managed Google account is present — the two things most likely to break if the binding or enrollment regressed. See **05 — Testing** for the exact pass/fail.

Gate 4 (pass/fail): For the piloted cohort — **devices manage under AMAPI, accounts add, apps deliver, devices compliant.** All PASS before expanding.

Rollback 4: If validation fails after migration, follow **06 — Troubleshooting**; if unresolved, halt further waves and escalate (Microsoft support) rather than pushing the failure to production.

Phase 5 — Expand (wave by wave)

Step 5.1 — Promote model by model, wave by wave. For each enrollment model that passed Pilot, migrate **Wave1**, validate (Phase 4), then **Wave2**, then **Prod** — re-running the validation report after each wave. Communicate the **verified** end-user impact to each wave **before** it migrates.

Step 5.2 — Keep tooling on hand. Retain `AMAPIMIG-Get-PostMigrationValidation.ps1` and the troubleshooting cross-reference for any recurrence as later waves migrate.

Gate 5 (pass/fail): Each wave validates (Phase 4 all PASS) before the next wave starts.

Rollback 5: Halt promotion; hold remaining waves on the current path. Already-migrated, validated devices are unaffected by halting future waves.

Order of operations (must not be reordered)

1. Binding healthy (R1) → 2. Estate inventoried (R2) → 3. **Current AMAPI guidance read** (R3) →
2. Wave planned + enrollment path decided (R5) → 5. **Pilot per model and OBSERVE re-enrollment /

impact **(R4)** → **6. Validate (manage under AMAPI, accounts add, apps deliver)** → **7. Expand wave by wave.** Skipping the pilot-observe step and promising an end-user outcome you did not verify is the primary failure mode this procedure prevents.**

Testing & Validation

Run these validations in order. Each has an **expected pass/fail** so you can prove a migrated cohort is healthy before promoting the next wave. The scripts referenced are described in **08 — Deliverables Summary**.

> ***Verification note.*** *Tests V2 and V5 depend on* ***current Microsoft guidance*** *for what a migrated (AMAPI) device and the correct enrollment/re-enrollment behavior look like. Read the current status surface — do not judge pass/fail against an assumed behavior.*

V1. Binding is healthy (R1)

How: `AMAPIMIG-Config.ps1 -HealthSummary`. **Pass:** Managed Google Play binding `bindStatus` is connected/bound and `lastAppSyncDateTime` is recent. **Fail:** Unbound / stale sync → repair the binding before migrating or interpreting any other result (a degraded binding fails app delivery downstream).

V2. Devices manage under AMAPI (the core outcome)

How: In the Intune admin center, confirm the migrated cohort reflects the post-migration (AMAPI) management state Microsoft's **current guidance** describes; corroborate device presence and check-in with `AMAPIMIG-Get-PostMigrationValidation.ps1 -GroupName "AMAPIMIG – <Model> – <Stage>"`. **Pass:** Devices are present, checking in, and shown in the AMAPI-managed state per current Microsoft guidance. **Fail:** Devices absent, not checking in, or still shown under the legacy state after the migration action → **verify against current Microsoft guidance**, then **06 — Troubleshooting**.

V3. End-user / re-enrollment impact matches the documented statement (R4)

How: Compare the **observed** pilot behavior (Phase 3) against the impact statement you committed for that enrollment model. **Pass:** The real user experience (no change / re-consent / re-enrollment / work-profile reset) **matches** what you documented and communicated for that model. **Fail:** The device did something you did **not** document (e.g. an unexpected re-enrollment or work-profile reset) → **stop expanding that model**, update the statement, re-communicate, and re-pilot before any further wave.

V4. Managed Google Play apps deliver

How: `AMAPIMIG-Get-PostMigrationValidation.ps1` reports managed-app install status for the cohort; confirm on a real device that a targeted managed Google Play app **installs**. **Pass:** Targeted managed Google Play apps install / remain installed on migrated devices (install status = installed for the required apps). **Fail:** Apps fail to install or disappear after migration → confirm V1 (binding) first, then app assignment against the wave group, then **06 — Troubleshooting**.

V5. Managed Google account adds / is present

How: On a migrated device (per model), confirm the managed Google account is present and app sign-in works; the validation report surfaces account/ownership signals. **Pass:** The managed Google account is present and managed Google Play functions on the device. **Fail — "can't add a Google account":** the classic Android enrollment failure → check network reachability to Google/Microsoft endpoints (Global Requirements), the binding (V1), and cross-reference the **Android enrollment troubleshooting (Google account / AMAPI)** solution.

V6. Devices remain compliant and Conditional Access passes

How: `AMAPIMIG-Get-PostMigrationValidation.ps1` reports `complianceState`; confirm a migrated user can still reach a CA-gated resource. **Pass:** Migrated devices are **compliant** and CA-gated access works. **Fail:** Devices non-compliant or users blocked after migration → check whether

re-enrollment reset compliance grace; review the CA/compliance policies noted in R6 before expanding.

V7. Wave repeatability

How: After Wave1 validates, migrate and validate **Wave2** the same way. **Pass:** Each subsequent wave produces the same PASS results as the pilot — confirming the process is durable, not a one-off. **Fail:** Inconsistent results across waves → capture the differing cohort and re-run the estate assessment + validation; do not promote to Prod until consistent.

Promotion gate: V1–V6 all **PASS** for a cohort before the next wave migrates; V2, V4, and V5 (**manage under AMAPI, apps deliver, accounts add**) are the non-negotiable trio. Re-run V1–V6 after each wave and after the final Prod cutover.

Troubleshooting

Post-migration troubleshooting pointers for the AMAPI migration. Match the symptom, apply the check, and re-validate with the referenced script. For device-level enrollment failures (especially **"can't add a Google account"**), this section deliberately **cross-references the dedicated Android enrollment-troubleshooting (Google account / AMAPI) solution** rather than duplicating it.

> ***Verify-first still applies here.*** *Whether a symptom is "expected AMAPI behavior" or a genuine fault (e.g. a re-enrollment prompt) is* ***version-dependent*** *— confirm the correct behavior against* ***current Microsoft guidance / your tenant*** *before treating it as a bug.*

Decision tree (start at the top)

```
After the migration action, is the device present and checking in?
■■ NO — device missing / not checking in                    → §1
■■ NO — user hit "can't add a Google account"               → §2 (see enrollment-troubleshooting solutio
■■ YES but not compliant / Conditional Access blocks user   → §3
■■ YES but managed Google Play apps did not (re)install     → §4
■■ Device re-enrolled / work profile reset UNEXPECTEDLY     → §5 (verify expected behavior first)
■■ Whole cohort/app-sync looks wrong (binding)              → §6

Admin can't see/select the wave group or devices (custom RBAC) → §7
Everything checks and it still fails                            → §8 (escalate)
```

Fast triage: run `AMAPIMIG-Get-PostMigrationValidation.ps1 -GroupName "<wave group>"` and `AMAPIMIG-Config.ps1 -HealthSummary` first — they localize most issues to a device, an app, or the binding.

§1 — Device missing / not checking in after migration

Symptom: A migrated device does not appear in the cohort, or `CheckInOk = False` in the validation report. **Checks:**

1. Confirm the migration action for that enrollment model was actually completed per **current Microsoft guidance** (Phase 3 / 5).

1. `AMAPIMIG-Get-PostMigrationValidation.ps1` — is `LastSync` stale? Force a device check-in and

re-run.

1. Network reachability to Google/Microsoft endpoints (Global Requirements) — a blocked endpoint

silently stops check-in. **Fix:** Restore reachability / re-run the documented migration step; if the device was expected to re-enroll, complete the re-enrollment per current guidance. Re-validate (V2).

§2 — "Can't add a Google account" during (re-)enrollment

Symptom: The device-side enrollment fails at adding the managed Google account. **Root causes (common):** blocked Google/Microsoft endpoints; a degraded managed Google Play binding; device-side account state. **Fix:** Confirm endpoints and the binding (`AMAPIMIG-Config.ps1 -HealthSummary`), then follow the **Android enrollment-troubleshooting (Google account / AMAPI)** solution, which owns the detailed symptom→cause tree for this failure. Re-validate (V5).

§3 — Device present but non-compliant / Conditional Access blocks the user

Symptom: Migrated device shows non-compliant, or the user is blocked by CA after migration. **Root cause:** A re-enrollment (where it occurs) can reset compliance grace; a CA/compliance policy reviewed in R6 may re-evaluate. **Fix:** Allow the compliance policy to re-evaluate / check-in; confirm the policies from R6 apply to the migrated state. Do not disable CA blindly. Re-validate (V6).

§4 — Managed Google Play apps did not (re)install

Symptom: Required managed Google Play apps are missing or `installState != installed` in the validation report. **Checks:**

1. `AMAPIMIG-Config.ps1 -HealthSummary` — is `lastAppSyncDateTime` recent and the binding bound? A

stale binding is the top cause.

1. Confirm the app assignment targets the **wave group** (or the migrated users/devices), per R6.
2. Allow normal Intune sync latency before concluding failure.

Fix: Repair the binding / correct the assignment, force a sync, re-run `AMAPIMIG-Get-PostMigrationValidation.ps1 -AppDisplayName "<app>"`. Re-validate (V4).

§5 — Device re-enrolled or work profile reset unexpectedly

Symptom: A device did something the impact statement did not predict (re-enrollment prompt, work-profile reset, or a dedicated-device reset). **First — verify it is actually unexpected.** Re-enrollment/reset behavior is **version- and model-dependent**. Confirm against **current Microsoft guidance** whether this is the intended behavior for that model at the current rollout stage. **If genuinely unexpected: Stop expanding that model**, update the impact statement (R4/V3), re-communicate to affected users, and re-pilot before any further wave. On dedicated/COSU, treat any reset as data-affecting and stage devices offline. **Fix / recovery:** Re-enroll the affected device clean per current guidance

(cross-reference the enrollment-troubleshooting solution) and record the true behavior so later waves are communicated correctly.

§6 — Whole cohort / app-sync looks wrong (managed Google Play binding)

Symptom: App delivery fails broadly, or the connection shows unbound/stale. **Check:** `AMAPIMIG-Config.ps1 -HealthSummary` — `bindStatus` and `lastAppSyncDateTime`. **Fix:** Repair or re-establish the managed Google Play binding (category Global Requirements / Android Enterprise connection solution) **before** interpreting any per-device result. A degraded binding masquerades as many migration bugs.

§7 — Admin can't see/select the wave group or devices (custom Intune RBAC)

Symptom: With custom Intune RBAC, the operating admin can't see the wave group or the migrated devices. **Fix:** Add the wave group to the admin's role-assignment **scope groups**, or operate with an admin whose scope includes them (tenants on built-in roles only are unaffected).

§8 — Everything checks and it still fails (escalation)

When binding is healthy, the estate is inventoried, the current guidance was followed, and validation still fails:

1. Capture the exact admin-center state and error, the enrollment model, device details, and the `AMAPIMIG-Get-PostMigrationValidation.ps1` / `-HealthSummary` output.
1. Re-confirm the correct current behavior against Microsoft's **live** Android Management API guidance (the rollout may have changed since you planned).
1. **Halt the affected wave** (do not push a failing migration to production) and open a **Microsoft support case** with the captured evidence.

Golden rule: because the AMAPI transition is evolving, a "failure" is only a failure once you have confirmed the correct current behavior. Verify against current Microsoft guidance, keep waves small, and let the pilot — not an assumption — tell you what the end user experiences.

Best Practices

Durable practices for migrating Intune Android Enterprise management to the **Android Management API (AMAPI)** with the least disruption.

Verify, never assume (the cardinal rule)

- **Read the current Microsoft guidance every time you plan or expand.** The AMAPI rollout, enrollment-method options, web/account-driven availability, and re-enrollment behavior change over time. Treat "verify against current Microsoft guidance / your tenant" as a hard gate, not a disclaimer.

- **Never promise an end-user outcome you have not observed.** Do not tell users "no re-enrollment" (or "you must re-enroll") until a pilot device of that exact enrollment model has shown you the behavior and Microsoft's current docs confirm it.
- **Determine legacy-vs-AMAPI from Microsoft's status surface, not from an inferred property.** There is no single documented Graph field that labels a device AMAPI-managed; read the authoritative status in the admin center / current guidance.

Estate and wave design

- **Assess before you plan.** Inventory by enrollment model, ownership, OS, OEM, and check-in first — waves and communications flow from the real population.
- **Pilot each enrollment model separately.** Work profile (BYOD), fully managed, corporate-owned work profile, and dedicated/COSU can each behave differently on migration; prove each on its own before mixing.
- **Sequence lowest-visibility first.** Migrate the cohorts with the least user-facing risk early; stage BYOD work-profile and dedicated/COSU (which can reset a kiosk) deliberately and late.
- **Keep wave groups Assigned, scoped, and single-purpose.** One Assigned security group per model/stage; don't reuse a group that also drives unrelated licensing or assignments.

Protect the managed Google Play binding

- **Confirm binding health first and after each wave.** AMAPI does not remove managed Google Play; a degraded binding shows up as app-delivery and account-add failures that masquerade as migration bugs.
- **Do not let the enterprise token/account lapse mid-migration.** Watch `lastAppSyncDateTime` and connection state throughout.

End-user communication

- **Communicate the verified impact, per model, before each wave migrates** — what will happen, when, and what (if anything) the user must do. A wave should never be surprised.
- **For dedicated/COSU, coordinate as a maintenance action.** If a re-provision/reset is possible, stage the device offline, not in front of a user or customer.

Change control, validation, and rollback

- **Pilot-first with pass/fail gates.** Prove *manage under AMAPI → accounts add → apps deliver → compliant* on the pilot before any production wave.
- **Halt-and-hold is the rollback.** If a wave fails or the impact is unacceptable, stop promoting

and hold remaining devices on the current path — already-validated devices are unaffected. There is no "un-migrate" button; the safeguard is small waves and a working pilot.

• **Validate with a real device, not just a successful admin action.** Confirm an actual managed app install and account presence, not merely that the console accepted the change.

Tooling and least privilege

• **Run read-only by default.** The assessment, readiness, and validation tools request read scopes; grant `Group.ReadWrite.All` only for the brief window you create wave groups with `-Commit`.

• **No secrets in automation.** All scripts use interactive/delegated `Connect-MgGraph`; store no app secrets or credentials.

Documentation and evidence

• **Record the verified impact statement, the current-guidance references, and the tenant-verified control paths** in your own runbook — labels and behavior are version-dependent.

• **Keep the assessment and validation output as a baseline** so a later regression is a one-command diff, and cross-reference the **Android enrollment troubleshooting (Google account / AMAPI)** solution for device-level failures.

Deliverables Summary

Documentation

Document	What it covers
01 — Solution Overview	What AMAPI is and how it differs from the legacy Play EMM API, the enrollment models affected (work profile / fully managed / corp work profile / dedicated-COSU), the web/account-driven enrollment path, the end-user / re-enrollment impact model, and the verify-first discipline.
02 — Requirements	Solution-specific prerequisites (healthy managed Google Play binding, estate inventory, **current AMAPI guidance read**, a **verified per-model impact statement**, web-enrollment readiness decision, enrollment-profile/app/CA review, pilot cohorts, Graph PowerShell) — each with Description / Why / Verify / Implement. References the category Global Requirements.

Document	What it covers
03 — Implementation	The ordered **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** migration procedure: inventory the estate, read current guidance, plan the wave, prepare groups/profiles, **pilot each model and OBSERVE re-enrollment/impact**, validate, then expand wave by wave — with pass/fail gates and a halt-and-hold rollback.
05 — Testing	Validation V1–V7 with expected pass/fail: binding healthy, **devices manage under AMAPI**, end-user impact matches the documented statement, **apps deliver**, **accounts add**, compliance/CA, and wave repeatability.
06 — Troubleshooting	Symptom → root-cause pointers for post-migration failures (devices not checking in, "can't add a Google account," apps not delivering, unexpected re-enrollment/reset, binding/app-sync, RBAC/scope) with the escalation path — cross-referencing the Android enrollment-troubleshooting solution.
07 — Best Practices	Verify-never-assume, estate/wave design, protecting the managed Google Play binding, end-user communication, change control/rollback, least-privilege tooling, and evidence.
Executive Summary	One-page outcome, approach, and why it is low-risk.

PowerShell (Microsoft Graph) tools

All scripts use modern-auth `Connect-MgGraph` (no stored secrets), request the **least** scopes, and are **read-only or safe-by-default**. None asserts AMAPI timelines or re-enrollment behavior — those are verified against current Microsoft guidance and observed in the pilot.

Script	What it does	Safe default
`AMAPIMIG-Assess-AndroidEstate.ps1`	Inventories enrolled Android devices by enrollment model, management agent, ownership, OS/OEM, compliance, and last check-in, with a stale-device list. Notes that legacy-vs-AMAPI is **not** a single Graph property — verify in the admin center.	Read-only

Script	What it does	Safe default
`AMAPIMIG-New-MigrationWavePlan.ps1`	Buckets the estate into Pilot/Wave1/Wave2/Prod cohorts per model (lowest-visibility first) and writes `AMAPIMIG-WavePlan.csv`; optionally creates the Assigned wave groups.	Read-only (group creation gated behind `-CreateGroups -Commit`)
`AMAPIMIG-Test-WebEnrollmentReadiness.ps1`	Checks prerequisite signals for web/account-driven enrollment (binding, device-owner enrollment profiles, endpoint reminders); explicitly flags that **availability must be verified against current Microsoft guidance**.	Read-only
`AMAPIMIG-Get-PostMigrationValidation.ps1`	For a wave cohort: devices check in, compliance state, ownership/account signals, and **managed-app install status** for a named app — the evidence that devices manage under AMAPI, accounts add, and apps deliver.	Read-only
`AMAPIMIG-Config.ps1`	Shared connection helper, naming/enrollment-type constants, and (`-HealthSummary`) the managed Google Play **binding health** report used before migrating.	Read-only

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

AMAPIMIG-Assess-AndroidEstate.ps1

AMAPIMIG-Config.ps1

AMAPIMIG-Get-PostMigrationValidation.ps1

AMAPIMIG-New-MigrationWavePlan.ps1

AMAPIMIG-Test-WebEnrollmentReadiness.ps1